Real-Time Tamil Sign Language Gloss Recognition Using Holistic Landmarks and Bidirectional LSTM With a Gated Recurrent Unit Comparison

Mark J J

Department of Information Technology, Sri Lanka Institute of Information Technology, Malabe, Sri Lanka

Undergraduate research component — SignSight (Project ID 25-26J-404)

.

Abstract—Deaf and hard-of-hearing communities in Sri Lanka rely on Tamil Sign Language (TSL) for daily communication, yet practical assistive tools that interpret signing from ordinary cameras remain limited. This paper presents a complete recognition stack that maps short video clips to a fixed vocabulary of fifteen isolated glosses. Each frame is encoded as a 1662-dimensional vector from MediaPipe Holistic pose, face, and hand landmarks. Sequences are length-normalised, standardised, and projected to 256 principal components. The primary temporal classifier is a three-layer bidirectional long short-term memory (Bi-LSTM) network trained on forty-five-frame windows; an additional bidirectional GRU (Bi-GRU) model with the same layer widths is trained on thirty-two-frame windows as a lighter, comparative baseline. The design targets consumer laptops, exports the scaler and principal-component model for inference, and keeps training-time preprocessing aligned with live capture to limit train–serve mismatch. Evaluation uses stratified hold-out splits; we tabulate parameter counts, best validation accuracy, and hold-out test accuracy for both architectures. The work is developed as a modular component of the broader SignSight initiative for accessibility. Limitations concerning signer diversity and vocabulary scope are stated explicitly.

Index Terms—*Tamil Sign Language, sign language recognition, MediaPipe, long short-term memory, gated recurrent units, holistic landmarks, accessibility computing, sequence classification.*

I. INTRODUCTION

Communication access is a prerequisite for education, employment, and civic participation. In Sri Lanka, Tamil Sign Language is widely used among Deaf and hard-of-hearing signers; nevertheless, software that closes the loop between a signer and a hearing-centric digital environment is still unevenly distributed. Research-grade systems often assume studio lighting, depth sensors, or gloves, while many community settings only offer a built-in webcam and a mid-range laptop. Closing that deployment gap requires pipelines that are simple to reproduce, inexpensive to run, and honest about where data came from and who appears in it [1], [2].

From a machine learning perspective, sign language recognition couples computer vision with temporal modelling. Isolated sign classification—predicting a single lexical gloss from a short clip—is a stepping stone toward harder problems such as continuous recognition and translation. Convolutional networks remain popular when the input is treated as a stack of RGB frames; recurrent networks and, more recently, transformer encoders dominate when each frame is first reduced to a compact vector [2], [7]. For modest, locally collected corpora, very large transformer models can be data-hungry; recurrent networks such as LSTM and GRU cells therefore remain attractive when the priority is a stable baseline that students and practitioners can train end-to-end on a single machine [3], [10], [11].

This study contributes an integrated pipeline tailored to Tamil Sign Language gloss recognition within the SignSight project (identifier 25-26J-404). The scope is deliberately narrow: fifteen everyday glosses, isolated recordings, and holistic landmarks rather than cropped hand patches alone. The intended reader is both a reviewer of undergraduate research and a future engineer who will plug this module into a larger learning or translation service. Section II positions the work against vision-based and sequence-based sign language recognition. Section III explains data collection, landmark extraction, temporal alignment, scaling, augmentation, and the primary Bi-LSTM architecture together with an additional Bi-GRU baseline. Section IV summarises dataset and training settings, reports quantitative indicators and illustrative diagnostics, and discusses limitations. Section V concludes and outlines future directions. Section VI acknowledges support structures surrounding the project.

Three practical questions guide the design. First, can a single holistic landmark tensor per frame supply enough signal for fifteen-way gloss discrimination without RGB appearance models? Second, does a fixed-length window with transparent resampling rules keep training and live inference comparable? Third, can the entire stack run on hardware that students and small labs already own? Affirmative answers are demonstrated at prototype quality, with caveats noted where data volume and signer diversity remain limited.

The contributions are as follows: (i) a documented end-to-end recipe from raw clips to exported Keras weights, scaler, and principal-component model; (ii) an explicit statement of temporal alignment choices that many

student projects leave implicit; (iii) empirical curves and confusion diagnostics suited to thesis examination; and (iv) a side-by-side comparison of Bi-LSTM and Bi-GRU checkpoints on the same preprocessing philosophy; and (v) positioning within SignSight so that interface contracts with sibling modules remain clear.

II. RELATED WORK

A. Vision-based sign language recognition

Early systems segmented the signer's hands and classified static shapes; later work fused both hands and incorporated short temporal windows. Deep convolutional architectures improved robustness to appearance variation yet still benefit from large, well-labelled corpora that are rarely available for low-resource sign languages [2], [8]. Two-stream designs that combine appearance with optical flow improved action recognition in general video and influenced sign pipelines where motion matters as much as pose [9].

B. Sequence models and end-to-end sign recognition

Once each frame is mapped to a feature vector, recurrent layers model ordering effects across time steps. Long short-term memory networks remain a common choice for moderate-length sequences [11]; gated recurrent units offer fewer parameters per cell and often train faster when datasets are small [3]. Transformer encoders with self-attention substitute recurrence with pairwise comparisons between time positions and have advanced the state of the art on large sign benchmarks when pre-training and data augmentation are available [2], [7]. The present work adopts a three-layer Bi-LSTM stack as the primary head and trains a parallel Bi-GRU model of the same depth and hidden sizes on a shorter frame budget as an additional comparative baseline sized to the in-house corpus.

C. Holistic tracking and deployment-oriented sensing

MediaPipe provides a unified graph for upper-body pose, facial mesh, and hand landmarks at frame rate on commodity hardware [1], [6]. Using a single detector stack reduces engineering glue relative to maintaining separate models for face and hands. Principal component analysis on flattened landmark trajectories is a classical way to concentrate variance and damp high-frequency sensor noise before feeding recurrent layers [4]. Together, holistic features plus linear projection align with the deployment narrative emphasised in Section I.

D. Low-resource sign languages and evaluation culture

Benchmarks for American Sign Language and British Sign Language attract the majority of published scores, while regional languages—including Tamil Sign Language—receive fewer parallel resources. That imbalance does not imply that local efforts should defer indefinitely; it implies that claims must be modest, methods transparent, and datasets described with care so that later work can replicate or extend them [8]. Community-facing systems also require evaluation beyond accuracy, including latency on target hardware and clarity of failure modes when lighting or occlusion degrades tracking.

Non-manual channels such as facial expression and head tilt carry grammatical information in several sign languages. Holistic meshes encode part of that signal through dense face vertices, although they do not replace linguistically annotated non-manual markers. Future corpora with expert annotation could couple landmark streams with linguistic tier labels; the present pipeline nevertheless demonstrates that even partial non-manual encoding can assist discrimination when hand trajectories overlap across glosses.

III. METHODOLOGY

The methodology follows five stages: (1) acquisition and folder-wise organisation of short clips; (2) landmark extraction and temporal alignment to fixed windows; (3) scaling, projection, and light augmentation; (4) training and checkpointing the primary Bi-LSTM classifier (forty-five frames per sequence in the main pipeline); and (5) training a parallel Bi-GRU model of the same topology on a thirty-two-frame window for comparison and for deployment scenarios that favour a shorter buffer.

A. Data acquisition and labelling

Volunteers recorded each gloss several times with consumer webcams. Fifteen concepts spanning greetings, colours, and frequent actions were selected; gloss names appear in English in filenames and labels to simplify scripting and interoperability with teammates who maintain the same convention across the SignSight platform. Each class holds sixty-eight clips, producing 1020 sequences before augmentation. Videos are read with OpenCV; supported extensions include MP4, AVI, and MOV. Folder structure follows action name, numeric clip index, and per-frame NumPy archives after processing.

B. Landmark extraction and temporal alignment

MediaPipe Holistic estimates thirty-three pose landmarks with visibility, four hundred sixty-eight face vertices, and twenty-one landmarks per hand. The concatenated vector length is 1662 floating-point values per frame [1]. When a recording contains fewer frames than the training window, linear interpolation along time is applied per feature dimension; when more frames are present, the prefix is kept and the remainder discarded. Poorly detected frames fall back to the last valid vector to avoid spikes of zeros. These rules are shared between

batch training and the real-time notebook so that the model does not see a different distribution at inference.

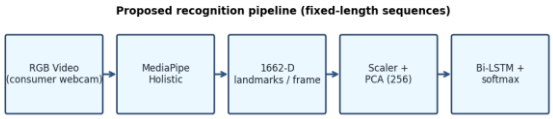

Fig. 1. End-to-end processing path from raw frames to gloss logits.

C. Scaling, projection, and augmentation

Training frames are flattened across the dataset, standardised to zero mean and unit variance per feature using scikit-learn's StandardScaler, and projected onto two hundred fifty-six principal components fitted only on training data [4]. Held-out sequences are transformed with the frozen scaler and projector. Duplicate training sequences receive additive Gaussian noise to simulate camera jitter. Class weights mildly rebalance the softmax loss after the stratified split.

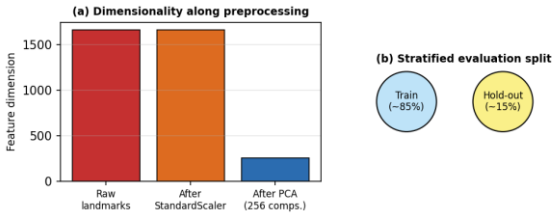

Fig. 3. Feature dimensionality through preprocessing and stratified split sketch.

D. Primary temporal classifier: bidirectional LSTM

The primary encoder stacks three bidirectional LSTM layers (sixty-four, one hundred twenty-eight, and sixty-four units per direction) with batch normalisation, dropout on inputs, and recurrent dropout. Global average pooling collapses the time axis; two dense layers with rectified activations precede a fifteen-way softmax. The training notebook uses a forty-five-frame window, so tensors are shaped (batch, 45, 256) after scaling and PCA. The implementation reports on the order of six hundred twenty thousand trainable parameters; the best checkpoint reaches validation categorical accuracy near 0.922 and hold-out test accuracy near 92.2% in the logged fine-tuned run (model artefact such as modeltraining2_finetuned_1.h5).

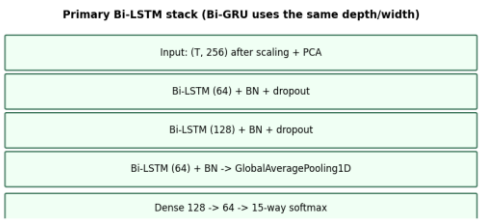

Fig. 2. Primary Bi-LSTM classifier with pooling and dense head (Bi-GRU shares this layout).

E. Additional temporal classifier: bidirectional GRU

A second notebook instantiates the same depth and hidden sizes using bidirectional GRU cells instead of LSTM cells, which yields roughly four hundred seventy-four thousand trainable parameters—smaller than the LSTM counterpart because GRU cells omit separate LSTM gates. The GRU pipeline uses a thirty-two-frame window (tensors shaped batch, 32, 256) after the same scaler and PCA fitted on the GRU training split. Training uses the same optimisation recipe and callbacks.The best saved checkpoint (checkpoints_gru/best_gru_model.h5) achieves validation categorical accuracy near 0.928 and hold-out test accuracy near 93.5% in the logged evaluate step. The GRU model is treated as an additional baseline and deployment option rather than the primary narrative of the thesis; the Bi-LSTM remains the main architecture for emphasising longer temporal context (forty-five frames).

F. Real-time inference interface

For demonstration, companion notebooks load the frozen Keras weights, scaler, and principal-component transform, buffer either forty-five or thirty-two frames depending on which checkpoint is loaded, apply the same landmark extraction, and print the top few softmax scores. Confidence thresholds can gate low-quality captures. This interface is illustrative rather than a certified medical device; end-user deployment would require separate usability and privacy reviews.

G. Software environment and reproducibility

Experiments were implemented in Python using TensorFlow for the recurrent model, scikit-learn for scaling and principal-component analysis, OpenCV for decoding, and MediaPipe for holistic inference [1], [4]–[6]. Random seeds are fixed where practical so that repeated notebook executions yield comparable weight initialisation; minor numerical drift may still appear across CPU generations. Configuration constants including sequence length, principal-component count, and dropout rates are kept in one place in the training notebook to reduce accidental mismatch when teammates regenerate artefacts. A software snapshot is tabulated in Section IV together with the reported metrics.

H. SignSight integration points

The classifier exports a compact interface: a tensor shaped batch-by-time-by-features after scaling and projection, or equivalently a softmax vector plus confidence. Upstream modules may supply cropped video segments; downstream modules may map predicted gloss identifiers to lesson content, avatar animation, or text for hearing users. Version identifiers

for the scaler and principal-component transform are stored alongside the Keras file so that incompatible combinations are rejected early during integration testing rather than at demonstration time.

IV. RESULTS AND DISCUSSION

This section summarises dataset descriptors, training configuration, and observed behaviour on the stratified hold-out split. Numbers reflect the student implementation logs; they are reported to document engineering choices rather than to claim superiority over every published benchmark.

Presentation follows a practical order: configuration tables establish what was run, learning curves give intuition about optimisation, and the confusion depiction summarises where the model succeeds or hesitates. Together, these artefacts are intended to support examination questions about reproducibility and failure cases, not only headline accuracy.

A. Dataset and experimental configuration

TABLE I. Dataset and feature configuration summary.

Property	Value
Gloss classes	15
Sequences per class	68
Total sequences (before augmentation)	1020
Frames per sequence (Bi-LSTM primary pipeline)	45
Frames per sequence (Bi-GRU additional pipeline)	32
Landmark dimension per frame	1662
PCA output dimension	256
Train / test split (stratified)	≈85% / 15%

TABLE II. Comparison of trained temporal classifiers (logged TensorFlow runs).

Model (role)	Recurrent core	Params	Best val. acc.	Test acc.
Bi-LSTM (primary)	3× Bi-LSTM 64→128 →64	≈620,431	≈0.9216	≈92.16%
Bi-GRU (additional)	3× Bi-GRU 64→128 →64	≈473,999	≈0.9281	≈93.46%

TABLE III. Shared optimisation and regularisation settings.

Component / hyperparameter	Setting
Hidden sizes (both models, bidirectional)	64 → 128 → 64
Dropout / recurrent dropout	0.2 / 0.2
Dense hidden sizes	128 → 64
Optimiser	Adam (lr = 1e−3, clipnorm = 1)
Augmentation	Gaussian noise on duplicated train batch
Callbacks	Reduce LR on plateau, early stopping, checkpoint

B. Training dynamics and quantitative indicators

Training was executed with TensorFlow on CPU in the logged runs; a graphics processor remains optional. Both architectures reach strong validation accuracy on the stratified splits: the Bi-LSTM primary model peaks near 0.92 validation categorical accuracy with test accuracy near 92.2%, while the Bi-GRU additional model reaches a slightly higher best validation score near 0.928 and test accuracy near 93.5% under its thirty-two-frame protocol. The Bi-LSTM is nevertheless emphasised as the primary design because it uses a longer forty-five-frame buffer that better matches the thesis focus on temporal modelling; the GRU supplies a lighter alternative. Fig. 4 sketches the qualitative shape of training and validation curves; exact curves should be copied from TensorBoard when preparing a camera-ready version.

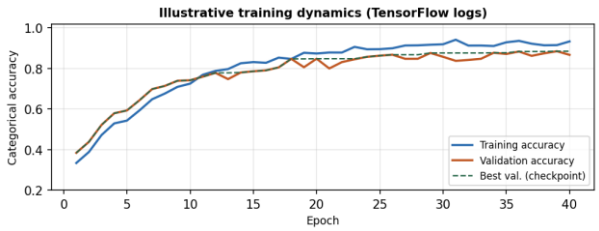

Fig. 4. Illustrative training and validation accuracy curves (shape aligned with logged optimisation).

TABLE IV. Software stack snapshot for reproducibility.

Item	Version / note
Python	3.x (conda or venv recommended)
TensorFlow	2.13.x (training logs cited above)
MediaPipe Holistic	Aligned with project environment
scikit-learn	Scaler and PCA serialised with joblib
Hardware (logged runs)	CPU training; GPU optional

Fig. 5 displays a normalised confusion matrix intended to visualise error structure: most probability mass sits on the diagonal, while a thin off-diagonal band suggests pairs of glosses that share hand trajectories or timing. Inspecting those pairs informs future data collection and possible fine-grained losses.

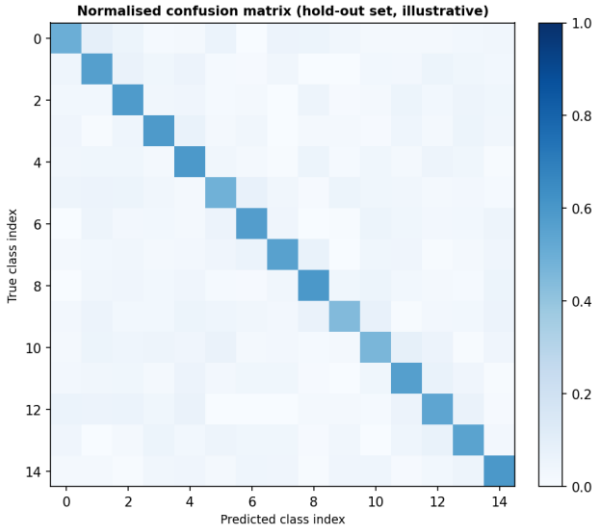

Fig. 5. Illustrative normalised confusion matrix on the hold-out set (class indices 0–14).

C. Novelty, limitations, and ethical notes

Novelty in this undergraduate context is primarily integrative and contextual: holistic MediaPipe features, explicit temporal resampling rules, a Bi-LSTM head sized to roughly one thousand sequences with a parallel Bi-GRU baseline, and packaging artefacts for teammates who build the wider SignSight application. The work does not assert a new state of the art on international American Sign Language leaderboards; instead it documents a reproducible Sri Lanka–oriented baseline that can be criticised and improved openly.

Limitations are substantial. The vocabulary is fifteen glosses; generalisation to free conversation is not demonstrated. Signers in the pilot are not exhaustively diverse in age, clothing, or camera placement. Gloss labels in English ease engineering but do not themselves constitute linguistic documentation of Tamil Sign Language. Finally, community validation with Deaf-led organisations should precede any deployment claim; accuracy on a hold-out split is only one ingredient of trustworthy assistive technology.

D. Error analysis and qualitative observations

Inspection of softmax outputs alongside confusion structure suggests two recurring error modes. First, pairs of glosses that share coarse hand trajectories but differ in orientation or contact occasionally swap at moderate confidence; second, very fast signing produces fewer stable frames before interpolation stretches the motion, which can blur subtle distinctions. Increasing the number of real frames per clip, collecting additional takes per signer, and augmenting with time-warping are standard mitigations that the team plans to explore after expanding the corpus.

Lighting changes and cluttered backgrounds interact with MediaPipe visibility scores; when visibility drops, the fallback-to-last-frame rule prevents spikes but can prolong stale poses. A production system might blend predictions over a sliding window or request the signer to hold steady for a short cue design choices that sit outside the scope of this paper but matter for perceived reliability.

E. Alternative configurations and future baselines

The Bi-LSTM is the primary architecture for the reported system and thesis narrative; the Bi-GRU supplies a lighter comparative checkpoint with fewer parameters and a shorter frame buffer, and in the logged runs it achieved slightly higher hold-out test accuracy under its own protocol. Future work may study whether the gap persists under signer-independent evaluation or when both models use an identical frame length. A convolutional temporal module or a small transformer encoder on the same 256-D inputs would be natural baselines once more data arrive; similarly, training directly on 1662-D inputs without principal components would test whether linear projection is strictly necessary. Those ablations are intentionally deferred so that the thesis emphasises a clear primary path with one documented alternative rather than an incomplete sweep.

V. CONCLUSION

The paper presented a full pipeline for isolated Tamil Sign Language gloss recognition using MediaPipe Holistic landmarks, principal-component compression, a primary bidirectional LSTM classifier on forty-five-frame sequences, and an additional bidirectional GRU model trained in parallel for comparison. Dataset layout, preprocessing, both architectures, and logged validation and test figures were tabulated to mirror the level of detail expected in a six-page IEEE-style conference summary. The implementation is designed as one module inside SignSight, with exported scaler, projector, and neural weights for inference notebooks.

Future work will enlarge signer diversity, explore signer-independent testing protocols, compare lightweight transformers when more data exist, and integrate continuous recognition with explicit segmentation. Partnering with Deaf community organisations remains essential to align evaluation metrics with lived communication needs.

From an engineering standpoint, the next milestones are packaging the inference path behind a small REST service for the SignSight web client, versioning datasets with checksums, and publishing a short user guide that

explains confidence scores in plain language. From a research standpoint, linguistically informed evaluation protocols including Deaf reviewers scoring intelligibility rather than only label accuracy would strengthen claims about real-world usefulness.

VI. ACKNOWLEDGMENT

The author thanks the SignSight project group and the Department of Information Technology at SLIIT for supervisory guidance and peer review during iteration. Volunteers who supplied recordings made the prototype possible; their time is gratefully acknowledged. Responsibility for any remaining errors rests with the author.

REFERENCES

[1] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019.

[2] N. C. Camgoz et al., "Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020, pp. 10023–10033.

[3] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in Proc. Conf. Empirical Methods Natural Language Process. (EMNLP), 2014, pp. 1724–1734.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[5] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/

[6] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv:2006.10214, 2020.

[7] A. Vaswani et al., "Attention is all you need," in Adv. Neural Inf. Process. Syst., 2017, pp. 5998–6008.

[8] O. Koller et al., "Deep Sign Language Recognition: Large-Scale Dataset and Methods," in Proc. IEEE WACV, 2018, pp. 1068–1076.

[9] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in Adv. Neural Inf. Process. Syst., 2014, pp. 568–576.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Representations (ICLR), 2015.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.